EJERCICIO 1

Una empresa guarda un listado de clientes en una cola de registros `[cod_cliente, activo]` donde el campo `activo` indica si el cliente sigue trabajando o no con la empresa. Implementar un subprograma que elimine de la cola aquellos clientes marcados como no activos.

Complejidad de tiempo esperada: O(n)
Espacio auxiliar esperado: O(n)

```
procedure eliminar(var c:tcolaRegistros);
var
  aux: tcolaRegistros;
  x: tCliente;
begin
    initialize_queue(aux);
    while not empty_queue(c) do { sacamos todos los elementos de la original }
    begin
      x:= first(c);
      dequeue(c);
      if x.activo then enqueue(aux,x); { guardamos solo los activos }
    end;
    while not empty_queue(aux) do { reintroducir activos en la cola original }
    begin
      x:= first(aux);
      dequeue(aux);
      enqueue(c,x);
    end;
end;
```

Reto: hacer con espacio de almacenamiento O(1)

```
(Pista) Utilizar 2 punteros, anterior y actual, para recorrer la cola de inicio a fin
```

EJERCICIO 2

Programa un procedimiento que muestre por pantalla la información de una cola que recibe como parámetro en orden inverso al de inserción.

Complejidad de tiempo esperada: O(n)
Espacio auxiliar esperado: O(n)

```
procedure mostrar_al_reves(var c:tcola);
var
  aux: tPilaEnteros;
  c2: tcola;
  x: integer;
begin
    initialize_stack(aux); { problemas con sobrecarga en Pascal: no admite 2
                             métodos initialize uno en cola y otro en pila }
    copy_stack(c,c2); { copia de la cola original para no modificarla }
    while not empty_queue(c2) do
    begin
      x:= first(c2);
      dequeue(c2);
      push(aux,x);
    end;
    while not is_empty(aux) do
      writeln(pop(aux));
end;
```

EJERCICIO 3

Programa una función que reciba una cola con números enteros distintos en un rango y retorne el elemento que falta o -1 si la secuencia es completa (incluye todos los enteros entre el menor y el mayor observados).

Complejidad de tiempo esperada: O(n)
Espacio auxiliar esperado: O(1)

```
ALGORITMO:

Sacarlos todos y a medida que sacamos calcular mayor, suma y menor. Al final el
número que falta será la suma teórica de todos (su suma es (mayor+menor)·(n+1)/2
menos (calculada por el sumatorio entre menor y mayor) y la suma calculada.

function elemento_falta(var c:tcola): real;
var
    n,min,max,actual,suma: integer;
begin
    n:=0;
    suma:=0;
    max:= -MAXINT;
    min:=  MAXINT;
    while not is_empty(c) do
    begin
      actual:= first(c);
      if actual>max then max:= actual;
      if actual<min then min:= actual;
      n:= n+1;
      suma:=suma+actual;
      pop(c);
    end;
    if (n=0) or (max-min+1 = n) then elemento_falta:= -1
    else elemento_falta:= (max+min)*(n+1)/2 - suma;
end;
```

EJERCICIO 4

Programa una función que reciba una cola que contiene números enteros en el rango de 1 a n y retorne una estructura de datos con todos los números que aparecen más de una vez y su frecuencia.

Complejidad de tiempo esperada: O(n)
Espacio auxiliar esperado: O(n)

```
ALGORITMO:

    (1) Array entre 1 y n, inicializado a ceros. Cada vez que sacamos uno sumamos 1 a
        su posición en el array. Al final recorrer el array y meter en una cola los
        que tengan frecuencia > 1.
    (2) Otras: lista ordenada, map/diccionario → O(n²). Hashmap → O(n)

function extraer_duplicados(var cola_original:tcola):tcola;
const
  MAX = 100; //Suponemos que el rango máximo será 1..100 (puede ser menor)
var
  cola_duplicados, cola_aux: tcola;
  i,x: integer;
  array_frecuencias: array[1..MAX] of integer;
begin
  initialize(cola_duplicados);
  initialize(cola_aux);
```

```
  for i:=1 to MAX do array_frecuencias[i]:=0;
  while not is_empty(cola_original) do
  begin   // Revisamos cada elemento y los pasamos a una cola auxiliar
    x:= first(cola_original);
    array_frecuencias[x]:= array_frecuencias[x] + 1; // anotar cada aparición de x
    enqueue(cola_aux,x);
    dequeue(cola_original);
  end;
  for i:=1 to MAX do // los repetidos (frecuencia>1) van a la cola de repetidos
    if array_frecuencias[i]>1 then enqueue(cola_duplicados,i);
  while not is_empty(cola_aux) do // Dejamos la cola original como estaba
  begin
    enqueue(cola_original,first(cola_aux));
    dequeue(cola_aux);
  end;
  extraer_duplicados:= cola_duplicados; // devolvemos la cola con los duplicados
end;
```

EJERCICIO 5

Programa un subprograma que reciba una pila y una cola de enteros e intercambie sus elementos sin alterar el orden, poniendo todos los elementos de la pila en la cola y todos los de la cola en la pila sin modificar el orden.

Complejidad de tiempo esperada: O(n)
Espacio auxiliar esperado: O(1)

```
ALGORITMO:

1 - Sacar todos los elementos de la cola y meterlos en la pila, contando en el
proceso cuántos elementos se han traspasado.

2 - Sacar la pila completa y llevar todos los elementos a la cola.

3 - Ahora sacar de la cola tantos elementos como contamos en el paso 1 para sacar
solo los que inicialmente estaban en la cola.

Tras esto, ya están en el orden original pero los de la pila en la cola y los de la
cola en la pila. Y sólo hemos empleado un entero como almacenamiento auxiliar.
```

EJERCICIO “Largo”

Un sistema genera procesos con una cierta prioridad (entre 1 y 3) y con un cierto nombre (string). Se pide lo siguiente:

1) Codificar un subprograma que genere n procesos de nombre y prioridad aleatoria y los meta en una cola que hará de buffer del sistema

```
Antes de nada, necesitamos implementar un TAD Cola de Procesos:

unit uColaProcesos;

interface
  type

    tProceso = record
      id: string;
      prioridad: 1..3;
    end;
```

```
    nodo = record
      info: tProceso;
      sig: ^nodo;
    end;

    tcolaProcesos = record
      first, last: ^nodo
    end;

  procedure initialize(var c: tcolaProcesos);
  function is_empty(c: tcolaProcesos): boolean;
  procedure enqueue(var c: tcolaProcesos; p: tProceso);

implementation

  procedure initialize(var c: tcolaProcesos);
  begin
    c.first := nil;
    c.last := nil;
  end;

  function is_empty(c: tcolaProcesos): boolean;
  begin
    is_empty:= (c.first = nil) and (c.last = nil);
  end;

  procedure enqueue(var c: tcolaProcesos; p: tProceso);
  var
    new_node: ^nodo;
  begin
    new(new_node);
    new_node^.info:= p;
    new_node^.sig:= nil;
    if is_empty(c) then
     begin
      c.first:=new_node;
      c.last:= new_node;
     end
    else
     begin
      c.last^.sig:=new_node;
      c.last:= new_node;
     end;
  end;
```

Una vez está listo, entonces procedemos a implementar el procedimiento que nos piden:

```
procedure generar_procesos(var cola:tColaProcesos; n:integer);
var
  i:integer;
  pproceso: ^tproceso;
begin
  randomize;
  for i:=1 to n do
    begin
      new(pproceso);
      str(random(1000),pproceso^.id);
      pproceso^.id:= 'P'+pproceso^.id;
      pproceso^.prioridad:= 1 + random(3);
      enqueue(cola,pproceso^);
```

```
    end;
end;
```

2) Implementar un subprograma que vacíe el buffer y genere 3 colas de procesos, una por cada nivel de prioridad, clasificando los procesos en dichas colas según su prioridad, pero respetando el orden de llegada original.

Primero hemos de ampliar el TAD ColaProcesos con las operaciones dequeue y first:

```
  function first(c: tcolaProcesos): tProceso;
  begin
    first:= c.first^.info;
  end;

  procedure dequeue(var c: tcolaProcesos);
  var
    aux: ^nodo;
  begin
    if not is_empty(c) then
    begin
      aux:= c.first;
      c.first := c.first^.sig;
      if c.first = nil then c.last:= nil;
      dispose(aux);
    end;
  end;
```

Una vez implementadas, procedemos a programar el procedimiento que nos piden:

```
procedure clasificar_procesos(var buffer,c1,c2,c3: tColaProcesos);
var
  p: tproceso;
begin
  while not is_empty(buffer) do
    begin
      p:= first(buffer);
      dequeue(buffer);
      case p.prioridad of
        1: enqueue(c1,p);
        2: enqueue(c2,p);
        3: enqueue(c3,p);
      end;
    end;
end;
```

3) Modifica el apartado 1 para que existan procesos de prioridad 0 (máxima prioridad) los cuales entran en el buffer como los demás, pero a la hora de clasificarlos (apartado 2) deben entrar en la cola por el principio saltándose todos los procesos de prioridad 1 que están en espera (este comportamiento convierte el TAD cola en un *TAD deque*).

Habría que modificar en la unit el tipo tProceso, así:

```
    prioridad: 0..3;
```

Y luego el procedimiento generar_procesos, así:

```
    pproceso^.prioridad:= random(4);
```

Y ya cuando tengamos esto, habría que añadir una operación "priority_enqueue" en el TAD ColaProcesos:

```
  procedure priority_enqueue(var c: tcolaProcesos; p: tProceso);
  var
    new_node: ^nodo;
  begin
    new(new_node);
    new_node^.info:= p;
    new_node^.sig:= c.first;
    if c.first = nil then c.last:= new_node;
    c.first:= new_node;
  end;
```

4) Implementar una función “in” que determine si un proceso de cierto nombre se encuentra en espera en el sistema y de estar, dónde (buffer, cola1, cola2 o cola3).

```
Primero habría que añadir a la unit la siguiente función:

  function in_queue(cola: tcolaProcesos; id_proceso:string): boolean;
  var
    encontrado: boolean;
    aux:^nodo;
  begin
    encontrado:= false;
    if not is_empty(cola) then
      begin
        aux:= cola.first;
        while (not encontrado) and (aux<>nil) do
        begin
          encontrado:= id_proceso = aux^.info.id;
          aux:= aux^.sig;
        end;
      end;
    in_queue:= encontrado;
  end;

Y una vez hecho esto, programar la función “in” que está enfocada a que devuelva un
entero que será 0 si aún no ha sido clasificada en su cola (está en buffer), 1, 2 o 3
si está en su cola y -1 si no está en el sistema. Algo muy sencillo, así:

function in_system(id_proceso: string; buffer, c1,c2,c3: tColaProcesos): integer;
begin
  if in_queue(buffer, id_proceso) then in_system:=0
  else if in_queue(c1, id_proceso) then in_system:=1
  else if in_queue(c2, id_proceso) then in_system:=2
  else if in_queue(c3, id_proceso) then in_system:=3
  else in_system:= -1
end;
```

5) Mostrar por pantalla todos los procesos que van a ejecutarse y su orden, según el siguiente formato (por simplificar supongamos que como mucho puede haber un proceso de máxima prioridad en el sistema en cualquier momento):

```
20 procesos en espera
1 de máxima prioridad
3 de prioridad 1
16 de prioridad inferior
```

```
procedure mostrar_procesos(var buffer,c1,c2,c3: tColaProcesos);
var
  p: tproceso;
```

```
  prioritarios: integer;
begin
  writeln(num_elems(buffer),' procesos en espera');
  if not is_empty(c1) then
    begin
      prioritarios:=0;
      p:= first(c1);
      if p.prioridad = 0 then prioritarios:=prioritarios + 1;
    end;
  writeln(prioritarios,' de maxima prioridad');
  writeln(num_elems(c1)-prioritarios,' de prioridad 1');
  writeln(num_elems(c2)+num_elems(c3),' de prioridad inferior');
end;
```